كيف تعمل لوحة PaySim Dashboard؟

دليل تشغيلي وتعليمي خطوة بخطوة، من فتح الصفحة حتى تتبع العملية والتنبيه.

1. ما هي Dash؟

Dash إطار عمل Python لبناء واجهات ويب تفاعلية. نحن نكتب المكونات في Python: Dropdown وGraph وDataTable وDiv. كل عنصر له ID وProperties. الـID هو اسم فريد نستطيع الإشارة إليه من Callback.

2. ما هو Callback؟

Callback دالة Python ترتبط بمدخلات ومخرجات. Input هو خاصية إذا تغيرت تستدعي الدالة. Output هو خاصية ستتحدث بنتيجة الدالة. State يقرأ قيمة دون أن يكون وحده سببًا لتشغيل الدالة. في مشروعنا عند اختيار Filter تتغير value، فيبدأ Callback ويطلب بيانات جديدة من SQL.

مثال split-filter.value → update_dashboard_analytics() → fetch_kpis() → SQL Server → KPI values → kpi-total.children.

3. الاتصال بين Dash وSQL Server

- تصل إشارة Input من المتصفح إلى Dash Server.
- Callback يستدعي وظيفة داخل data_access.py.
- data_access يبني WHERE clause ومعاملات Parameters.
- pyodbc يفتح الاتصال بقاعدة PaySimFraudDB.
- SQL Server ينفذ SELECT ويرجع Rows.
- Pandas DataFrame يستقبل النتيجة.
- الدالة تحوّل DataFrame إلى Plotly Figure أو Records صالحة لـDash.
- Dash يعيد Output إلى المتصفح فيتحدث العنصر دون إعادة تحميل الصفحة كاملة.

4. شرح ملفات التطبيق

الملف	المسؤولية
src/dashboard/app.py	إنشاء Dash app وتجهيز Assets وتسجيل Callback modules وتشغيل WSGI server.
components/layout.py	بناء العناصر المرئية والـIDs: KPIs وGraphs وDropdowns وTables.
components/charts.py	بناء Plotly Figures والتحقق من وجود الأعمدة والتعامل مع Empty Data.
queries/data_access.py	تنفيذ الاستعلامات على SQL Server ومعالجة الفلاتر والحدود.
callbacks/dashboard_callbacks.py	تحديث KPIs والرسوم وحالة الاتصال والوقت والأخطاء.
callbacks/transaction_callbacks.py	البحث وتحديث جدول العمليات وبناء لوحة تفاصيل الصف المحدد.
callbacks/alert_callbacks.py	تحديث جدول التنبيهات بناءً على الفلاتر.
assets/theme.css	الألوان والتنسيقات responsive styles للواجهة.

5. العناصر الفعلية وIDs

الوظيفة / IDs	المنطقة
split-filter, type-filter, risk-filter	الفلاتر
refresh-button, refresh-interval	التحديث
kpi-total, kpi-fraud, kpi-fraud-rate, kpi-high, kpi-medium, kpi-f1	KPIs
type-chart, risk-chart, temporal-chart	الرسوم
db-status, dashboard-error, last-updated	حالة التطبيق
transaction-search, transaction-table, transaction-details	البحث والجدول
alert-status-filter, alert-type-filter, alert-table	التنبيهات

6. كيف تقرئين المؤشرات الستة؟

- Total Transactions: عدد العمليات في نطاق الفترة ونوع العملية.
- Actual Fraud: عدد القيم isFraud=1 المخزنة للتقييم. هذا Label حقيقي من PaySim، وليس تنبؤًا.
- Fraud Prevalence: Actual Fraud ÷ Total Transactions × 100.
- High Risk: عدد السجلات ذات RiskLevel HIGH / high-risk count وفق استعلام KPI.
- Manual Review: عدد السجلات MEDIUM.
- F1 Score: المتوسط التوافقي بين Precision وRecall؛ في الشاشة الحالية يعكس نطاق Query المحدد، والافتراضي ALL يجمع Validation وTest.

تحذير عرض Dashboard default F1 = 99.52% محسوب على بيانات Validation + Test المجمعة في استعلام المؤشرات. Final Test F1 المستقل = 99.3569%. عند العرض الأكاديمي استخدمي القيمة الأخيرة عند الحديث عن الاختبار النهائي.

7. كيف تقرئين الرسوم الثلاثة؟

- Fraud by Transaction Type: أعمدة Actual Fraud وPredicted Fraud، وخط Total Transactions على محور ثانوي.
- Risk Distribution: Donut لأعداد العمليات داخل LOW/MEDIUM/HIGH؛ ليست احتمالات.
- Fraud Activity Over Time: Volume bars + actual fraud line + predicted fraud line + 7-step moving average.

8. Transaction Explorer خطوة بخطوة

- اختاري Split وType وRisk.
- اكتبي رقم SourceRowNumber أو Sender ID أو Receiver ID داخل Search.
- دالة fetch_transactions تنفذ Query بحد 100 صف.
- Dash DataTable تعرض 14 عمودًا وتسمح بالفرز والاختيار.
- عند اختيار الصف، `selected_rows` يطلق Details Callback.

• يقرأ الـCallback بيانات الصف من State ثم يبني Panel تفاصيل العملية.

• RawDecisionScore يظهر كرقم خام، وليس probability.

9. Alert Center خطوة بخطوة

• الفترة ونوع العملية يأتيان من الفلاتر العامة.

• Alert Status: ALL أو OPEN أو UNDER_REVIEW أو RESOLVED.

• Alert Category: ALL أو HIGH_RISK أو MANUAL_REVIEW.

• fetch_alerts يرجع 15 حقلًا من Alerts مع سياق العملية والقرار.

• التاريخ يتحول إلى ISO string وDecimal إلى float وBoolean إلى YES/NO.

• يعرض الجدول حتى 100 تنبيه في كل مرة.

10. التحديث التلقائي

dcc.Interval مضبوط على 30 × 1000 millisecond، أي 30 ثانية. عند انتهاء الفترة، يطلق n_intervals Callback. زر Refresh Data يطلق n_clicks. كلاهما يطلب قراءة جديدة للبيانات. لا يعني التحديث أن النظام يعيد تدريب الموديل أو يعيد تحميل CSV؛ هو يعيد استعلام SQL.

11. لماذا لا تتغير كل الأرقام مع كل فلتر؟

كل Query له Parameters خاصة به. fetch_kpis تقبل dataset_split وtransaction_type فقط. fetch_type_analysis تقبل dataset_split وrisk_level. fetch_risk_distribution وfetch_temporal_trend تقبلان dataset_split وtransaction_type. fetch_transactions تقبل split/type/risk/search. fetch_alerts تقبل split/type/status/type. لذلك Risk Level لا يغير جميع الرسوم والمؤشرات في النسخة الحالية.

12. ماذا يحدث عند الخطأ؟

الـAnalytics Callback يلتقط Exception ويعيد placeholders ورسالة خطأ إلى dashboard-error. Transaction وAlert Callbacks تسجل الخطأ في logger وتعيد جداول فارغة. هذا يمنع توقف الواجهة بالكامل، لكنه يعني أن المطور يحتاج متابعة logs لتشخيص تفاصيل الفشل.

13. ما الذي لا تفعله لوحة التحكم؟

• لا تتصل بحسابات أو معاملات بنكية حقيقية.

• لا تنفذ Payment Authorization.

• لا توقف أو تعكس تحويلًا حقيقيًا.

• لا تعرض Raw Decision Score كاحتمال.

• لا تعيد تدريب الموديل مع كل Refresh.

• لا تقدم server-side pagination لكل 280,613 صفًا؛ كل Query يعيد عينة bounded من 100 صف افتراضيًا.

14. تنبيه صلاحية خصائص الأرصدة

تحذر من استخدام الأرصدة قبل/بعد العملية في اكتشاف الاحتيال لأن Kaggle على PaySim ملاحظة مصدر المعاملات المكتشفة كاحتيال تُلغى. وبما أن النموذج الحالي استخدم خصائص مشتقة من الأرصدة، يجب فهم

لوحة التحكم باعتبارها عرضًا لنتائج التجربة الحالية لا دليلًا نهائيًا على الأداء القابل للنشر. الخطوة التالية هي تدريب نسخة أخرى بدون حقول الأرصدة أو الخصائص المشتقة منها، ومقارنة النتائج قبل اعتمادها.